Introduction to Artificial Intelligence

Mihir Rao

Contents

1 Introduction to AI **2**
- 1.1 Rational decisions . . . 2
- 1.2 Agents and environments . . . 2
- 1.3 Methods and applications . . . 3

2 Uninformed Search **4**
- 2.1 Planning and search problems . . . 4
- 2.2 State graphs and search trees . . . 4
- 2.3 Depth-first and breadth-first search . . . 5
- 2.4 Uniform-cost search . . . 5

Lecture 1: Introduction to AI

1.1 Rational decisions

Artificial intelligence can be approached through four goals: making machines think like people, act like people, think rationally, or act rationally. This course focuses on the last: choosing actions that best achieve specified goals. A decision can be rational even if it does not resemble human thinking.

A rational decision maximizes expected utility, a numerical measure of the desirability of an outcome. If h denotes the information available when choosing an action, O is the set of possible outcomes, and $U(o)$ is the utility of outcome o, then for a finite outcome set the rule is

$$a^* \in \arg\max_a \sum_{o \in O} P(o \mid h, a)\, U(o).$$

For each action, weight each possible outcome's utility by its probability, then choose an action with the largest total. Rationality is relative to the goals and information available at decision time: an unlikely bad outcome can occur even when the chosen action had the highest expected utility. The criterion evaluates the decision, not whether its internal reasoning imitates a person.

1.2 Agents and environments

An agent perceives its environment and acts on it. Sensors deliver percepts, and actuators carry out actions. The environment then changes and supplies new percepts, so decisions form a continuing interaction. The nature of the observations, available actions, and environment determines which techniques are appropriate.

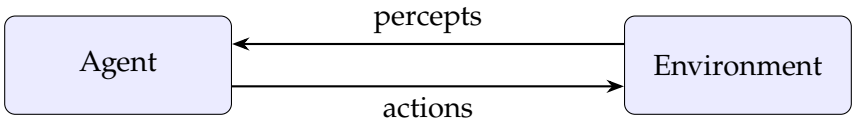


Figure 1: An agent interacts with its environment.

For Pac-Man, the environment is the game, percepts describe what the agent can observe, and actions move Pac-Man through the maze. The agent's objective specifies what counts as a good outcome; seeing the same game does not by itself determine what the agent should prefer.

A REFLEX AGENT chooses from its current percept, possibly using memory, without explicitly evaluating future consequences. A PLANNING AGENT uses a model to consider how candidate actions might change the world and selects a plan in light of its goal. A reflex rule can still be rational when it selects an optimal action for the setting; planning is one way to make decisions, not the definition of rationality.

1.3 Methods and applications

AI systems combine computation and data. Search and planning compare possible actions; constraint satisfaction finds assignments that obey specified rules; adversarial and uncertain settings require reasoning about other agents or uncertain outcomes. Probabilistic models represent uncertainty, decision theory connects uncertainty to utility, and machine learning uses data to improve predictions or decisions.

The same principles appear in different applications:

- **Language:** speech recognition maps audio to text, speech synthesis maps text to audio, and language systems support translation, question answering, and text classification.
- **Vision:** perception turns pixels into information, such as object identities, scene labels, or three-dimensional structure.
- **Robotics:** perception, planning, and control connect decisions to physical actions. The real world can differ from a simulator.
- **Games and logic:** game-playing systems reason over moves; logical systems use deduction, constraint satisfaction, and satisfiability to solve structured problems.

The field has used both knowledge-based and statistical approaches. Recent systems in the 2018 course draw on large datasets, computation, and neural networks; the appropriate representation and objective still depend on the task.

Source. Pieter Abbeel and Dan Klein, Introduction to Artificial Intelligence, Fall 2018, Lecture 1 and Note 1.

Lecture 2: Uninformed Search

2.1 Planning and search problems

A planning agent considers possible consequences before acting. It needs a model of how actions change the world and a goal test. A plan can be computed in advance or recomputed as new observations arrive. Search evaluates possible plans in the model, so its conclusions depend on the model's accuracy.

A search problem specifies a state space, a start state, a goal test, and a successor function. For a state, the successor function gives the available actions, resulting states, and step costs. A solution is a sequence of actions taking the start state to a goal; its cost is the sum of step costs. An optimal solution has minimum cost among all solutions.

A search state is an abstraction of the world: it retains the information needed to determine successors, costs, and goal status. For pathfinding in a maze, a position may suffice. To eat every dot, the state must also record which dots remain. If there are n possible positions and k dots, this representation has at most $n2^k$ states. Adding irrelevant details enlarges the search; omitting a detail that affects future actions or the goal makes the model inadequate.

2.2 State graphs and search trees

In a state-space graph, each state is represented once, and an edge represents an action and its successor. A search tree unfolds the possible plans from the start: its root is the start and each child extends its parent's plan by one action. A tree node represents a path as well as its end state. Thus different nodes can end at the same state, and a cycle in a finite state graph can generate an infinite search tree. Both structures are usually constructed only as needed.

Tree search maintains a fringe of partial plans. It removes a node according to an exploration strategy, checks whether its state is a goal, and otherwise expands it by adding successor plans. The strategy determines which plans are considered first. A goal is returned when selected from the fringe, which is especially important when path costs differ.

We compare strategies by completeness, whether a solution is found when one exists, and optimality, whether the returned solution has least cost. Let b be the maximum branching factor, m the maximum tree depth, and s the depth of the shallowest goal. These tree-search bounds count nodes, including repeated states; assume finite b.

2.3 Depth-first and breadth-first search

Depth-first search (DFS) expands a deepest fringe node first, using a last-in, first-out stack. It can pursue one branch indefinitely, so tree search is not complete for an infinite-depth tree. With finite depth it takes $O(b^m)$ time in the worst case. Its fringe holds at most the unexplored siblings along a root-to-leaf path, requiring $O(bm)$ space. DFS need not find a shallowest or least-cost solution.

Breadth-first search (BFS) expands a shallowest fringe node first, using a first-in, first-out queue. It explores by depth and is complete when branching is finite and a solution exists at finite depth. It finds a shallowest goal, which is optimal when all step costs are the same positive constant. Its time and space are exponential in solution depth; a conservative bound for the usual goal test on removal is $O(b^{s+1})$, because expansions at depth s before the goal can generate depth $s + 1$ nodes.

Iterative deepening runs depth-limited DFS repeatedly with increasing limits. It finds a shallowest goal with the space advantage of DFS. Although it revisits upper levels, for $b > 1$ most nodes lie near the deepest level, so the repeated work changes the asymptotic time by only a constant factor.

2.4 Uniform-cost search

When step costs differ, the fewest actions need not yield the cheapest plan. Uniform-cost search (UCS) uses a priority queue ordered by the cumulative path cost g. It removes a cheapest partial plan, tests for a goal, and expands it if necessary. It explores in increasing cost contours, without using information about the direction of the goal.

Suppose branching is finite, every step costs at least $\epsilon > 0$, and an optimal goal has finite cost C^*. UCS is complete: only finitely many tree nodes can have cost at most C^*, since their depth is at most C^*/ϵ. It is also optimal. If a more expensive goal were removed first, a frontier prefix of an optimal path would have cost at most C^* and would have been selected before that goal. This argument relies on nonnegative step costs and on checking the goal when it is removed, not merely when it is generated. Time and space are exponential in the effective depth C^*/ϵ, with implementation-dependent boundary factors from generating the next level.

Source. Pieter Abbeel and Dan Klein, Introduction to Artificial Intelligence, Fall 2018, Lecture 2.